RESEARCH BRIEF · FOR DISCUSSION WITH PROFESSOR

Does compressing what a vision-language model "sees" make it harder to audit?

The Compress-under-a-Safety-Constraint (CSC-VLM) project: the question, the foundations, the mathematics, what we have measured so far, and what we must check next.

Prepared by	Arya (TSEC Mumbai / IIT Madras BS) with research partners
Date	10 November 2026
Target venues	ICML 2027 (primary), CVPR 2027 (secondary)
Status	Measurement phase. Results are **preliminary**; the full cluster runs are still pending.
Scope of this brief	Ideas, math, metrics, evidence and decisions. Code and debugging details are left out on purpose.

Contents

1. One-page summary
2. The big picture in plain words
3. Foundations from first principles: vectors, features and superposition
4. Foundations: attacks, gradients and the smallest perturbation
5. Foundations: monitors, probes and how we score them
6. The original hypothesis: the causal chain, hypotheses and rival explanations
7. Our first metric, and why we had to throw it away
8. The current metric: obfuscation cost and its three-part split
9. Two competing forces: coverage versus interference
10. Experimental design
11. Toy-model results
12. First real-model results
13. What the results mean, and the working reframe
14. Loopholes we must close before trusting any result
15. Decision rules and next steps
16. The longer-term goal: compression under a safety constraint (ADMM)
17. Questions for the Professor
18. Glossary and references

1 · One-page summary

> **The question**
> Vision-language models (VLMs) turn an image into hundreds of "visual tokens". Many recent papers throw most of these tokens away to make the model faster. Separately, safety researchers build *monitors* that read a model's internal activations to catch bad behaviour. We ask: **if you compress the visual tokens, does it become cheaper for an attacker to change the model's answer without the monitor noticing?**

Original idea. Fewer tokens means the same visual information is squeezed into less space. Squeezed information overlaps ("superposition"). Overlap is known to make models easier to attack.[3,4] So compression should make monitors easier to dodge.

What we have learned so far.

1. Our first measurement ("safety score S") was broken. It mostly measured noise. All eight early runs carry no evidence either way.
2. We replaced it with a clean, per-image quantity: the **obfuscation cost (OC)**, the size of the smallest input change that flips the answer while keeping the monitor unchanged. It splits exactly into three parts: *how sure the model is*, *how sensitive it is*, and a *monitor tax*.
3. Working from first principles, we found that the original hypothesis is **not guaranteed**. Two forces pull against each other: **coverage** (a small monitor sees a larger share of a smaller space) and

interference (overlap makes outputs easier to move).

4. A trained toy model and the first real runs (Qwen2.5-VL-3B, SmolVLM-500M) both show compression **making the monitor see more, not less**, while the model becomes less robust through information loss and higher sensitivity. That is the **opposite** of our original hypothesis.

> **Working reframe (not yet confirmed)**
>
> "Visual-token compression changes robustness and monitorability *separately*, and often in opposite directions. The monitor's size (rank) and how subtle the concept is decide which way the monitor effect goes." This could explain why published papers disagree on whether pruning helps or hurts safety.

Before we trust any sign, four loopholes must be closed: (a) measure in one fixed space at every compression level (this alone can flip the result for pooling); (b) check our linear formula against a real attack; (c) use a concept that is not trivially easy; (d) use the same model in both stages.

What we need from you: feedback on the reframe, the choice of a "fair" monitor across budgets, the threat model, and the effect size that matters in practice (Section 17).

2 · The big picture in plain words

2.1 What a vision-language model does

A VLM (for example LLaVA[20], Qwen2.5-VL[21] or SmolVLM[22]) answers questions about images. It has three parts:

1. A **vision encoder** cuts the image into small square patches and turns each patch into a list of numbers (a vector). Each vector is a **visual token**. A typical image gives several hundred tokens.
2. A **projector** converts those vectors into the "language" the text model understands.
3. A **language model** reads the visual tokens plus the question, and writes the answer one word at a time.

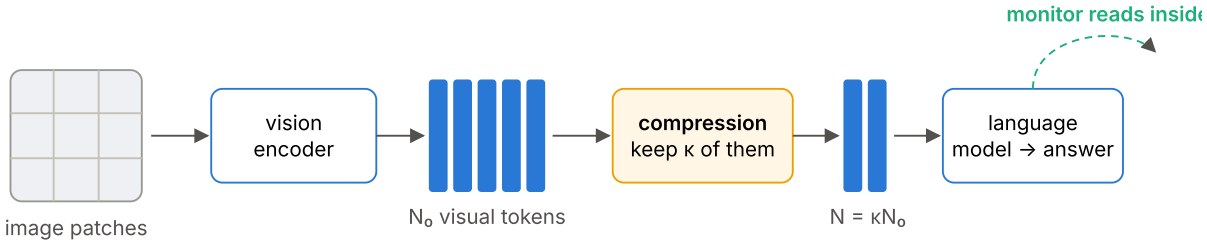


Figure 1. A VLM with visual-token compression. The monitor reads the language model's internal state, not just its final answer.

2.2 Why people compress visual tokens

The language model's cost grows with the number of tokens it reads. Visual tokens are often the majority. Methods such as FastV[11], SparseVLM[12], VisionZip[13], DivPrune[14] and token merging (ToMe)[15] report that 70–90% of visual tokens can be removed with little accuracy loss. These papers measure accuracy and speed. They do not measure what happens to the model's internal structure.

2.3 Why people monitor model internals

An attacker can sometimes make a model give a harmful or wrong answer. A **monitor** is a small detector that reads the model's hidden activations and raises an alarm. Examples are HiddenDetect for multimodal jailbreaks[16] and the "refusal direction" in language models.[17] The idea is that the inside of a model is harder to fake than its output. But work on language models has shown that a clever attacker can change the output *and* fool such monitors at the same time.[18]

2.4 Our question, precisely

> **Research question**
> At equal clean accuracy, does a VLM with fewer visual tokens let an attacker flip its answer with a **smaller** input change while a monitor stays fooled? And if so (or if not), **why**?

We call the property we care about **auditability**: how much a monitor forces an attacker to pay extra. If compression lowers auditability, every efficiency paper is quietly trading safety for speed. If it raises it, that is also important and surprising. The project is designed so that either answer is publishable, as long as we can explain it.

3 · Foundations: vectors, features and superposition

3.1 Vectors, directions and overlap

Inside a model, every piece of information is a **vector**: a list of *d* numbers (d is about 2,000–4,000 in our models). Think of it as an arrow in a *d*-dimensional space. Two arrows **u** and **v** can be compared with the **dot product** and the **cosine**:

$$u \cdot v = u_1 v_1 + u_2 v_2 + \ldots + u_d v_d \,, \quad \cos(u,v) = u \cdot v \,/\, (\|u\|\,\|v\|) \tag{3.1}$$

Here $\|u\| = \sqrt{(u \cdot u)}$ is the length of the arrow. A cosine of 0 means the arrows are at right angles (**orthogonal**): changing one does not affect your reading of the other. A cosine near 1 means they point the same way and are hard to tell apart.

Projection. The part of **v** that lies along a direction **u** (unit length) is $(u \cdot v)u$. The part left over, $v - (u \cdot v)u$, is "perpendicular to u". We will use this idea constantly: a monitor sees the part of a change that lies in its directions, and is blind to the perpendicular part.

3.2 Features

A **feature** is a property of the input the model tracks, such as "there is a dog" or "this is fur texture". The standard working assumption is that each feature corresponds to a direction in activation space, and an activation is roughly a weighted sum of the active features' directions:

$$h \approx z_1 v_1 + z_2 v_2 + \ldots + z_F v_F \tag{3.2}$$

where v_i is the direction of feature i and z_i is how strongly it is present. Most z_i are zero for any given image (features are **sparse**).

3.3 Superposition

If there are more features F than dimensions d, they cannot all be at right angles. Elhage et al.[1] showed that networks still store them, by letting the directions overlap slightly. This is called **superposition**. It works because sparse features are rarely active together, so the overlap ("interference") usually does no harm.

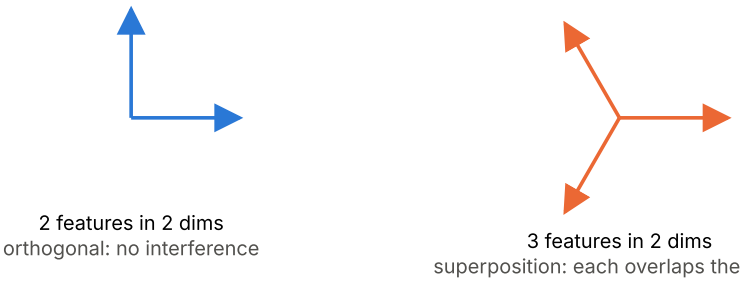


Figure 2. Superposition: storing more features than there are dimensions forces the directions to overlap.

3.4 How much overlap is forced? The Welch bound

Welch[2] proved a hard limit. If you place F unit-length directions in an m-dimensional space with F > m, then at least one pair must overlap by at least

$$\mu_{max} = \max_{i \neq j} |\cos(v_i, v_j)| \geq \sqrt{[\,(F - m) \,/\, (\,m\,(F - 1)\,)\,]} \tag{3.3}$$

For example, with F = 100 features in m = 50 dimensions the worst overlap is at least √(50/4950) ≈ 0.10; in m = 10 dimensions it is at least √(90/990) ≈ 0.30. **Fewer dimensions means more forced overlap**, no matter how cleverly you arrange things. This was the original reason we expected compression to matter: fewer tokens means fewer dimensions to store the same features.

3.5 Why overlap could help an attacker

Stevinson et al.[3] and Gorton & Lewis[4] showed in toy models and small vision models that adversarial attacks exploit interference between superposed features. Roughly, the attacker's best perturbation for moving the answer from class j to class k points along $W^\top(w_k - w_j)$, which is built from the overlaps. When features were made orthogonal, attacks stopped working. Liu et al.[5] further link superposition to how models scale.

3.6 Measuring "how many dimensions are really used"

Activations rarely use all d dimensions equally. The **participation ratio** (PR) is a simple count of the effective number of dimensions. If $\lambda_1, \lambda_2, \ldots$ are the variances along the main axes of the data (the eigenvalues of its covariance matrix):

$$\mathrm{PR} = d_{\mathrm{eff}} = (\Sigma\, \lambda_i)^2 \,/\, \Sigma\, \lambda_i^2 \tag{3.4}$$

If all k used axes have equal variance, PR = k. If one axis dominates, PR ≈ 1. We also use **r90**, the number of axes needed to explain 90% of the variance. The proposal defined **superposition pressure** as $\rho = F_{\mathrm{eff}} / (N \cdot d_{\mathrm{eff}})$: features per available dimension.

4 · Foundations: attacks, gradients and the smallest perturbation

4.1 An adversarial attack

An **adversarial attack** adds a small, carefully chosen change δ to the input so that the model's answer changes.[6] We measure the change by its length $\|\delta\|$. A model is **robust** when only large changes can flip its answer.

4.2 The behaviour score b and the margin m

For a yes/no style question (for example "cat or dog?"), the model gives a score (a *logit*) to each answer word. We define the **behaviour score**

$$b = \text{logit}(\text{positive answer}) - \text{logit}(\text{negative answer}) \tag{4.1}$$

If $b > 0$ the model says "positive"; if $b < 0$ it says "negative". The **margin** $m = |b|$ is how far the model is from changing its mind. A larger margin means more confidence. (We combine several spellings of each answer word, such as " cat" and "Cat", so the score does not depend on one token.)

4.3 The gradient: how sensitive the score is

The **gradient** $g = \partial b / \partial v$ is a vector that says how much b changes when each input number v is nudged. For a small change δ, a first-order (straight-line) approximation is

$$b(v + \delta) \approx b(v) + g \cdot \delta \tag{4.2}$$

A long gradient $\|g\|$ means the score is very **sensitive**: small input changes move it a lot.

4.4 The smallest perturbation that flips the answer

To flip the answer we need $g \cdot \delta = -b$. The shortest δ that does this points straight along the gradient. Its length is

$$\|\delta^*\| = |b| \,/\, \|g\| = m \,/\, \|g\| \tag{4.3}$$

This is the idea behind DeepFool[7] and the Carlini–Wagner attack[8]. It shows that the cost of an attack has two ingredients: **how sure** the model is (m) and **how sensitive** it is ($\|g\|$).

4.5 Real attacks: PGD, and why we must check our formula

Equation 4.3 is exact only if the model were a straight line. Real models are curved. The standard real attack is **projected gradient descent (PGD)**[6]: take many small gradient steps, keeping δ inside a size limit. Two well-known traps exist:

- **Gradient masking.** Some operations (such as "keep the top-k tokens") have zero or misleading gradients. Attacks then look weak, and a model looks falsely robust. Athalye, Carlini and Wagner found that 7 of 9 defences at ICLR 2018 relied on this effect and were broken.[9]
- **Weak attacks.** A poorly tuned attack also fakes robustness. Best practice is to use many steps, restarts and gradient-free checks.[10,27]

This is why our plan includes a mandatory "instrument check": compare our formula with the true smallest PGD perturbation on a sample of images (Section 14).

5 · Foundations: monitors, probes and how we score them

5.1 A linear probe

A **linear probe**[19] is the simplest monitor. It takes the hidden activation h at a chosen layer and position and computes one number, c·h, along a direction c. If the number crosses a threshold τ, it raises an alarm. Our canonical probe uses the **class-mean difference**:

$$c = \text{mean}(h \mid \text{positive images}) - \text{mean}(h \mid \text{negative images}), \quad \tau = \text{midpoint of the two class means} \tag{5.1}$$

This has no tuning knobs, so there is nothing to argue about. It reads the **last token** (where the answer is about to be produced) at about 70% of the model's depth. A probe with r directions instead of one is called **rank r**. Deployed probes are usually rank 1.

5.2 Refit versus frozen monitors

- **Refit:** retrained at every compression level. This measures how detectable the behaviour is in principle. It is our headline.
- **Frozen:** trained on the uncompressed model and reused. The gap between refit and frozen measures **staleness**, a practical deployment risk.

5.3 Scoring a detector: AUROC and d′

AUROC[23] is the probability that the monitor scores a random positive example higher than a random negative one. 0.5 means guessing; 1.0 means perfect separation. **d′** (d-prime) measures the same separation as the distance between the two class means divided by their spread. A concept whose AUROC is already ≈1.0 is **saturated**: it cannot show any decline, so it is a poor test case.

5.4 The monitor-aware attack

A clever attacker wants to change b *without* changing what the monitor reads. If G is the matrix of the monitor's gradients (one row per monitor direction), the attacker solves:

$$\text{minimise } \|\delta\| \;\text{ such that }\; b + g\cdot\delta = 0 \;\text{ and }\; G\delta = 0 \tag{5.2}$$

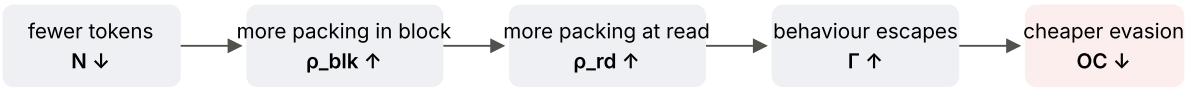
In words: flip the answer, move the monitor by exactly zero, and use the smallest possible change. The difference between this cost and the plain attack cost (Equation 4.3) is the extra price the monitor forces. That price is what we call auditability. Section 8 solves this problem exactly.

> **Why a monitor is essential to the study**
> Without a monitor, we would only be studying robustness (already widely published) or superposition (a different paper). A "perfect" monitor with unlimited capacity reduces to how well the two classes separate, which is just information. Auditability is a separate quantity only for **capacity-limited** monitors, which is what real deployments use. So every claim we make must state the monitor rank r.

6 · The original hypothesis

6.1 The causal chain

The proposal proposed one chain of cause and effect. Each arrow is a claim that can be tested:



all measured at matched clean accuracy

Figure 3. The original causal chain. Γ (the "evasion gain") is the share of the behaviour's direction that the monitor cannot see.

6.2 Pre-registered hypotheses

ID	Claim	Supported if (proposal thresholds)
H1	Compression raises superposition	ρ rises as N falls on most models and measures; feature count F_{eff} grows sub-linearly (exponent $\alpha \leq 0.6$)
H2	Compression lowers obfuscation cost	OC falls ≥ 25–40% across the budget range at matched accuracy
H2b	Merging vs pruning	Merging pays a similar tax (superposition) rather than a smaller one (information loss)
H3	Γ mediates the effect	Γ explains ≥ 50% of the total effect (mediation analysis[24])
H4	Causal test	Lowering ρ at fixed N (anti-superposition fine-tune) raises OC; a placebo fine-tune does not
H5	Universality	Random pruning shows the same effect (it is about *how many* tokens, not *which*)

6.3 Four rival explanations we must rule out

ID	"It is not superposition, it is..."	How we separate it
R1	**Information loss:** the compressed model simply knows less, so its margin is smaller	The margin term log m is reported separately; matched-accuracy control; pool vs merge_dup
R2	**Selection steering:** the effect comes from *which* tokens survive	Random-drop null; query-based vs norm-based selection
R3	**Gradient masking:** discrete selection hides gradients and fakes robustness	Real PGD and gradient-free attacks; the instrument check
R4	**Attention sinks:** a few special tokens absorb attention[25,26]	Compare which tokens are kept (norm vs random); layer/position maps

6.4 Gates

The plan has three go/no-go checkpoints. **G1:** does random pruning show the effect? **G2:** is the attack strong and not gradient-masked (ratio of PGD cost to gradient-free cost within 0.8–1.25)? **G3:** does an intervention move ρ first and then OC, with a placebo that does nothing? We do not scale up past a gate that fails.

7 · Our first metric, and why we had to throw it away

7.1 What it was

The first version computed a geometric "safety score":

$$a = \Sigma^{1/2} w_b, \quad \Gamma = \|P_\perp a\| / \|a\|, \quad S = \beta / (\Gamma \cdot \|a\|) \tag{7.1}$$

Here w_b is a behaviour direction, Σ is the covariance of the activations, $P_\perp$ removes the monitor's directions (w_b plus top principal components), and β is a target shift. Γ was estimated from only 24 images in a 4,096-dimensional space.

7.2 What was wrong

1. **The answer was fixed in advance.** The behaviour direction w_b was itself one of the monitor's directions, so true evasion was zero by construction.
2. **Γ was mostly noise.** With 24 samples in 4,096 dimensions, the estimate is dominated by sampling error. At no compression, Γ was 0.004 on the training images but 0.33–0.63 on new images. In a simulation where the true value is 0, the method reported $\Gamma \approx 0.81$.
3. **Unpaired design.** Different random seeds were used at each compression level, so differences mixed with chance.
4. **Labels were ignored**, and for pruning the readout only looked at the kept high-norm tokens, which collapsed the reading.
5. One "check ratio" was equal to 1 by algebra, so it could never fail.

7.3 What the eight early runs really showed

Runs 1–8 covered LLaVA-7B/13B × cats/person × merge/prune. Under merging, the "escape" $\Gamma \cdot \|a\|$ stayed roughly constant (e.g. 1.15 → 1.33). Γ appeared to rise only because $\|a\|$ shrank (correlation between them −0.41 to −0.95). Under pruning, the whole reading collapsed (1.15 → 0.16), so S moved the wrong way. The predicted effect failed in 8 of 8 runs.

> **Conclusion**
> None of Runs 1–8 is evidence for or against the hypothesis. The metric was broken, not the idea. We also found that the old "merge" did not really reduce the number of tokens: it averaged groups but kept every position, so it reduced information while the count stayed the same. It is now kept only as a control called *merge_dup*.

7.4 The lesson that shaped everything after

Every metric must come from a clearly defined question (an optimisation problem or a probability), not from an ad-hoc ratio. It must be computed per image, use labels, keep the behaviour separate from the monitor, and be checkable against a real attack. The new metric in Section 8 was built to these rules.

8 · The current metric: obfuscation cost and its three-part split

8.1 Setting up

For each image we compute three things at the compressed visual tokens v (the "carriers"):

- b, the behaviour score, and the margin m = |b| (Equation 4.1);
- g = ∂b/∂v, the gradient of the behaviour (Section 4.3);
- G, the matrix whose rows are the gradients of the monitor's reading(s) with respect to v.

8.2 Solving the monitor-aware attack exactly

We solve Equation 5.2: flip b while keeping the monitor exactly still. Split g into two parts: the part inside the space the monitor can see (the span of G's rows), and the part outside it:

$$g = g_{\parallel} + g_{\perp}, \quad g_{\perp} = (I - P_G)\, g \tag{8.1}$$

Any δ that keeps the monitor still must satisfy Gδ = 0, so it lives entirely outside the monitored space. There, only $g_{\perp}$ can move b. So the attack is the "smallest flip" of Equation 4.3, but using $g_{\perp}$ instead of g:

$$\delta^* = -b \cdot g_{\perp} / \|g_{\perp}\|^2, \quad \|\delta^*\| = m / \|g_{\perp}\| \tag{8.2}$$

(One can check: Gδ* = 0 because $g_{\perp}$ is outside the monitored space, and g·δ* = −b because $g \cdot g_{\perp} = \|g_{\perp}\|^2$.)

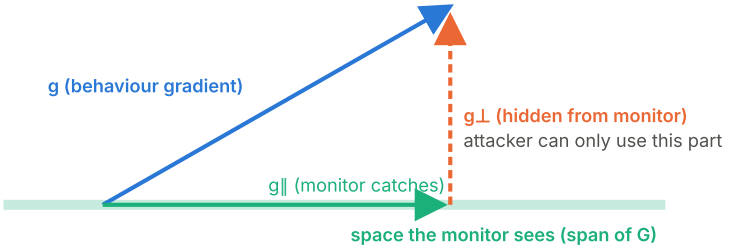


Figure 4. The behaviour gradient splits into a part the monitor sees and a part it does not. The larger the share the monitor captures, the less the attacker has to work with.

8.3 Capture, the monitor tax and the decomposition

Define the **capture** as the fraction of the gradient's squared length the monitor sees, and the **monitor tax** T:

$$\text{cap} = \|g_{\parallel}\|^2 / \|g\|^2, \quad \|g_{\perp}\| = \|g\| \sqrt{(1 - \text{cap})}, \quad T = 1 / \sqrt{(1 - \text{cap})} \tag{8.3}$$

Then the obfuscation cost becomes

$$\mathbf{OC = m \cdot T / \|g\|} \tag{8.4}$$

Taking logarithms turns the product into an exact sum, per image:

$$\log \text{OC} = \log m \; - \; \log \|g\| \; + \; \log T \tag{8.5}$$

Term	Meaning in plain words	What it tests
log m	How sure the model is. Falls if compression throws away information.	Rival R1 (information loss)
−log ‖g‖	How hard it is to move the output at all. Falls if the model becomes more sensitive.	Ordinary robustness
log T	The extra price the monitor forces. T = 1 means the monitor adds nothing; T = 2 doubles the attack cost.	**Auditability: the actual claim**

> **Why this split matters**
> OC can fall for three very different reasons. Only a fall in T means "compression makes monitors weaker". A fall in m just means the model knows less, and a fall in −log‖g‖ means it is less robust in general. The old metric mixed these together; the new one separates them exactly for every image.

8.4 Extra measurements that make the result interpretable

- **Matched placebo.** A random direction that the monitor captures by the *same* amount as the behaviour's read gradient. If T rises equally for the placebo, the rise is generic geometry, not about our behaviour.
- **Three-way split** of the alignment A = −log(1 − cap): A = *bypass* + *specific* + A_{match}. *Bypass* is influence that travels through routes the last-token monitor cannot see (other positions, later layers). *Specific* is what is special about this behaviour beyond the matched placebo.
- **Full-capacity detector cost** $|b| / \sqrt{(c^T \Sigma c)}$: the best any detector reading this position could do. It is an upper bound that does not depend on monitor rank.
- **Incidental detection:** does the canonical probe already flag the plain (monitor-unaware) attack?
- **Rank sweep:** every result is reported at r = 1 (headline) and r = 2, 4, 8, because the sign can depend on r.

8.5 How sure are we that it is computed correctly?

The gradients agree with finite-difference checks (relative error 2×10^{-5}); the probe and the full-capacity cost match independent recomputation; the placebo matches the behaviour's capture as designed; all 22 automated tests pass. **High confidence** that the code computes the formula. **Unknown** whether the formula equals the true smallest attack in a curved VLM (margins of ~6 logits are a long straight-line step). **Medium confidence** that "monitor exactly unchanged" is the right constraint: a real monitor only needs to stay below its threshold, so real evasion could be cheaper.

9 · Two competing forces: coverage versus interference

Working from first principles, we realised that compression does two things at once, and they push the monitor tax T in opposite directions.

9.1 Force 1: coverage (makes monitors stronger)

Imagine the behaviour's gradient points in a random direction in an m-dimensional space, and the monitor sees a fixed r-dimensional slice of it. On average the monitor captures

$$\mathrm{E}[\mathrm{cap}] = \mathrm{r} / \mathrm{m} \quad \Rightarrow \quad \mathrm{E}[\Gamma^2] = \mathrm{E}[1 - \mathrm{cap}] \approx 1 - \mathrm{r} / \mathrm{m} \tag{9.1}$$

If compression shrinks the effective space m while the monitor keeps the same rank r, the monitor sees a **larger share**. The attacker has less hidden room, so T goes **up**. Example: a rank-8 monitor in a 200-dimensional effective space sees about 4%; in a 20-dimensional space it sees about 40%.

9.2 Force 2: interference (makes monitors weaker)

Squeezing features together makes them overlap more (Welch bound, Section 3.4). The behaviour can then "hitch a ride" on overlapping directions the monitor does not read, and the model's outputs become easier to move. This pushes T **down** and also raises $\|g\|$. This was the force our original hypothesis relied on.



Figure 5. Which force wins decides the sign of the monitor effect. Our data so far say coverage wins.

9.3 What decides the winner

- **Monitor rank r compared with the effective dimension.** Coverage depends on r/m; a richer monitor gains more from a smaller space.
- **How subtle the concept is.** A strong, obvious concept (cats vs dogs) is easy for any monitor. A subtle concept spread thinly across many features is where interference may win.

This is why the reframe in Section 13 says the sign of the effect is predicted by these two quantities. Turning this into a testable proposition (coverage term versus interference gain) is one of the theory pieces we need for a main-track paper.

10 · Experimental design

10.1 Compression operators

Name	What it does	Fewer tokens?	Role in the study
pool	Average neighbouring patches on the image grid (2-D pooling)	Yes	Merge-family method; keeps most information
drop_norm	Keep the tokens with the largest length	Yes	A common pruning rule
drop_random	Keep a random subset	Yes	**Null baseline** (G1): "how many" vs "which"
drop_query	Keep tokens most similar to the question at an early layer	Yes	Query-based pruning (rival R2)
merge_dup	Replace each group by its mean but keep all positions	**No**	Control: less information at the same count
readout_only	Run the full model; only the monitor's reading is restricted	No	Control for reading artefacts

We report compression by the **keep fraction** κ (for example 1, 1/2, 1/4, 1/8, 1/16). All "drop" methods really shorten the sequence.

10.2 Models, data and concepts

- **Models so far:** Qwen2.5-VL-3B[21] (Stage 0) and SmolVLM-500M[22] (Stage 1 partial). Earlier runs: LLaVA-1.5-7B/13B.[20] Planned: at least four model families.
- **Concepts:** cats vs dogs[28] (easy; saturated), "bottle present" from COCO[29] (small objects; not saturated), and later a real safety concept such as jailbreak images from FigStep[30] or MM-SafetyBench.[31]

10.3 Stages

Stage	What it measures	Cost
Toy (X1, X1b)	Checks that our estimators recover known truth	Minutes, CPU
Stage 0	Forward pass only: accuracy, margin, probe AUROC, staleness, packing (PR, r90)	< 1 GPU-hour per model
Stage 1	Gradients: OC, its three terms, placebo, bypass/specific split, full-capacity cost	A few GPU-hours
Stage 2	Real PGD and gradient-free attacks in pixel space (validity check)	Larger; only if justified

10.4 Statistical rules

- **Paired design:** the same images at every keep level, so each image is its own control.
- **Slope:** $\beta = d \log X / d \log \kappa$, the per-image slope of a quantity X against the keep fraction. $\beta > 0$ means X **falls** under compression. Read it as "percentage change in X per percentage change in

tokens kept".

- **Uncertainty:** 95% confidence intervals from bootstrap resampling of images (or training seeds), [32] never from slopes of a few medians.
- **Nulls:** matched placebo, random drop and readout_only in every run.
- **One fixed space:** the attacker's perturbation space must be identical at every keep level (Section 14.1).
- **No pre-committed direction:** either sign is a result if the decomposition explains it.

11 · Toy-model results

A toy model is a tiny network where we know the true answer, so we can check whether our measurements are honest.

11.1 X1: a hand-built toy (estimator checks only)

- It confirmed the old Γ estimate is biased by noise: 0.41 measured vs 0.26 true at 24 samples.
- It confirmed our closed-form OC equals an explicit attack in a linear model (expected, since the toy is linear).
- It showed that discrete token selection can hide cheaper attacks (a "masking index" of 2.4, well outside the safe 0.8–1.25 band). This matters for Stage 2.

> **Limitation**
> X1 builds the mechanism in by hand, so it would agree with our hypothesis by design. We use it only to check estimators, never as evidence.

11.2 X1b: a trained toy where structure emerges on its own

Here a small network is trained, and superposition appears naturally. We tested two regimes over 3 training seeds.

Regime	What happened to the monitor tax T	Reading
Carrier bottleneck	T **rises** with compression at rank 1, for every method, in 3/3 seeds. The matched placebo rises too.	Generic **coverage** geometry, not behaviour-specific
Subtle behaviour	For pool and drop_random, T **falls** at rank 1 but **rises** at rank 4.	**The sign depends on monitor rank.** Pool beats the matched null; drop_random's change is mostly information loss

In the first quick run we could also see both forces at once for pool: capture in the monitor's read space fell (0.95 → 0.79) while capture in carrier space rose; coverage won overall. The key lesson: **the original hypothesis can be true in some regimes (subtle concept, small monitor) and false in others**. That is exactly the structure the reframe relies on.

12 · First real-model results (preliminary)

12.1 Stage 0: Qwen2.5-VL-3B, cats vs dogs

256 evaluation images, 144 for training the monitor, read at layer 25, fp16 precision.

Method	Accuracy at keep 1/16	Margin slope β
pool	0.941	0.61
drop_norm	0.898	0.67
drop_random	0.961	0.32
drop_query	0.891	0.52
merge_dup	1.000	0.32
readout_only	1.000	0.00 (clean control)

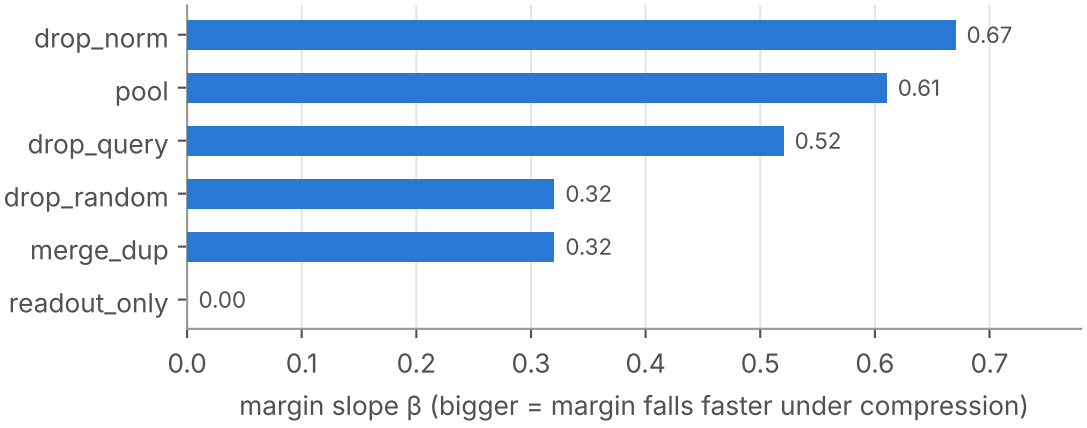


Figure 6. Margin falls under compression for every real method (information loss, rival R1). The control shows zero, as it should.

What Stage 0 tells us

- **Information loss is real** for every method, yet accuracy stays ≥ 0.89 even with 1/16 of the tokens.
- **The concept is too easy.** Refit monitor AUROC ≥ 0.993 everywhere, so AUROC cannot show any decline.
- **Which tokens matter:** random keeping beats keeping the largest-norm or most question-similar tokens at extreme compression. This hints at attention-sink effects (R4). Query-based selection gave no advantage here.
- **No sign of extra packing.** The block participation ratio falls from 23 to 6–7 for *every* method, including the readout_only control. PR divided by the number of rows stays ≈ 0.5 throughout. So the fall is simply because there are fewer rows, not because tokens are more tightly packed. The read-position r90 stays at ~22–27. **The first link of the original chain has no support.**
- **Staleness** appeared only for drop_norm (frozen monitor AUROC 0.960 vs 0.993 refit).

Caveat: fp16 overflow warnings appeared; we will confirm in bf16 on the cluster.

12.2 Stage 1: SmolVLM-500M (partial)

48 images, fp32, rank-8 refit monitor; merge_dup only at keep 1. Medians, keep 1 → 1/16.

Method	Margin |b|	‖g‖	Capture	Tax T	OC change
pool	6.33 → 3.38	0.40 → 0.51	0.35 → 0.80	1.24 → 2.21	×0.75
drop_random	falls (log m −0.49; −log‖g‖ −0.23)		0.35 → 0.67	1.24 → 1.75	×0.69
drop_norm	falls (log m −0.33); −log‖g‖ +0.16 (less sensitive)		0.35 → 0.51	1.24 → 1.43	≈ flat

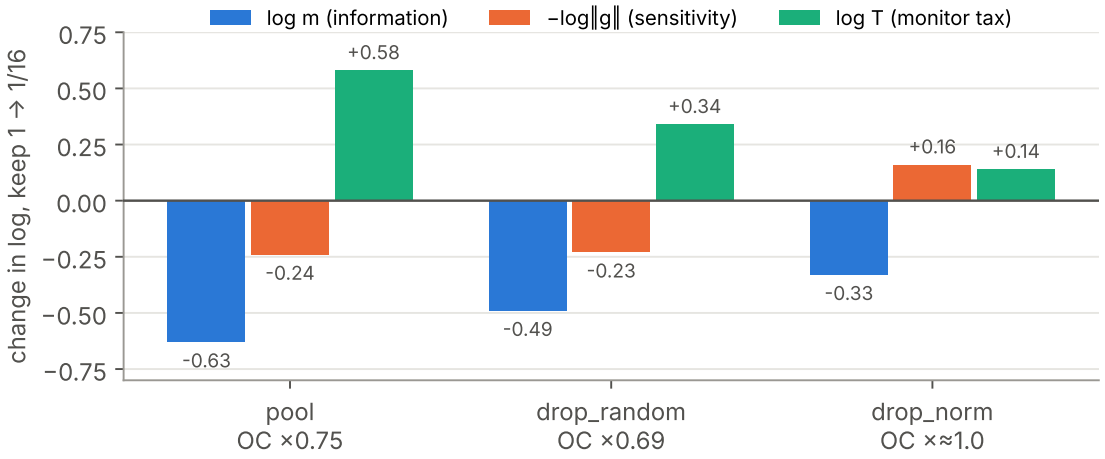


Figure 7. The three parts of the change in log OC (Equation 8.5). Where OC falls, information (blue) and sensitivity (orange) drive it. The monitor term (green) pushes the other way.

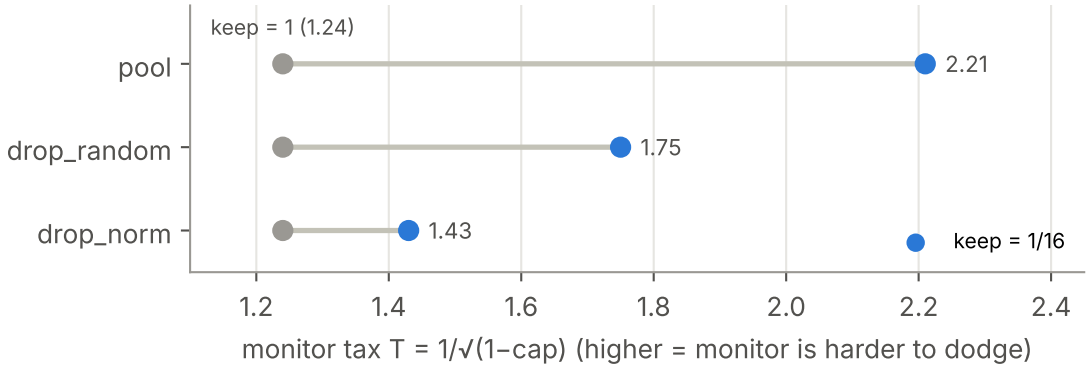


Figure 8. The monitor tax rises under compression for every method: the monitor sees more.

- Almost all of the change happens at keep ≤ 1/8. There is a "knee": mild compression is nearly free.
- We have not yet checked whether the T rise is behaviour-specific (matched placebo and bypass split are in the saved results but not yet analysed).
- Stage 0 and Stage 1 used different models, so they cannot be chained into one story yet.

13 · What the results mean, and the working reframe

13.1 Reading against the original hypothesis

Link	Prediction	Evidence so far
N ↓ → packing ↑	Tokens become more tightly packed	Not supported: the fall in PR is a row-count effect; read r90 flat
packing → Γ ↑ (monitor sees less)	Capture falls, T falls	Opposite so far: capture and T rise (toy and SmolVLM)
OC ↓	Cheaper evasion	Sometimes, but driven by information loss and sensitivity, not the monitor term; and the sign may flip in a fixed space (Section 14)
Robustness ↓	(not the claim)	Yes: margins fall for every method

13.2 The candidate claim

> **Candidate paper claim**
> "Visual-token compression trades robustness for monitorability. Information loss and higher sensitivity make attacks cheaper, while coverage makes simple monitors see more. The sign of the monitor effect is predicted by monitor rank versus effective dimension, and by concept subtlety. This explains the conflicting results on pruning and safety in the literature."

The published, output-only results already disagree: our notes record an ICML 2026 token-pruning study reporting that pruning mostly hurts robustness but query-based pruning helps, and a 2026 preprint (arXiv:2607.22716) reporting that the right pruning *improves* robustness.[33] Output-only checks tell us whether the answer flipped, but they cannot separate the rival causes or measure monitors. Our three-level design (outcome → monitor under attack → mechanism) is aimed exactly at that gap.

13.3 What a main-track paper would need

1. A **predictive rule**: a cheap geometric statistic that predicts the sign before running attacks, confirmed out of sample.
2. At least **four VLM families** × real compression methods + the random null.
3. At least one **real safety monitor and task** (e.g. jailbreak detection).
4. **Pixel-space PGD** confirmation.
5. A **theory piece**: the coverage proposition (Equation 9.1) against an interference gain, tested on the trained toy.

Honest assessment: today's evidence supports a workshop paper. With items 1–5, it becomes a credible main-track submission.

14 · Loopholes we must close before trusting any result

14.1 The measuring space (critical)

Stage 1 measures the attack in "carrier space": the compressed tokens themselves. But the number of carriers changes with keep, so a length $\|\delta\|$ means different things at different keep levels. For pooling this matters a lot.

Suppose one carrier is the average of k patches: $c = (p_1 + \ldots + p_k)/k$. By the chain rule, the gradient with respect to each patch is $\partial b/\partial p_i = (1/k)\, \partial b/\partial c$. Adding up over the k patches:

$$\|g_{\text{patch}}\|^2 = k \cdot (1/k)^2\, \|g_{\text{carrier}}\|^2 = \|g_{\text{carrier}}\|^2 / k \;\Rightarrow\; \text{OC}_{\text{patch}} = \sqrt{k} \cdot \text{OC}_{\text{carrier}} \tag{14.1}$$

The tax T does not change (for equal groups), and drop methods are not affected. But at keep 1/16, $k \approx 16$, so $\sqrt{k} \approx 4$. Pool's OC change goes from **×0.75 to about ×3**: **the sign reverses**. Intuitively, averaging acts like a smoothing defence: the attacker must push 16 patches to move one carrier. **Fix:** report OC in the fixed space of pre-compression patch embeddings (and in pixels for Stage 2), alongside carrier space.

14.2 Is the straight-line formula realistic?

Margins are around 6 logits, so the linear formula takes a long step. We will run a **mini PGD instrument check**: about 32 images × keep {1, 1/4, 1/16} × {pool, drop_random}, in the fixed space. **Pass** if the rank correlation between our formula and the true smallest perturbation is ≥ 0.3 and their ratio stays in a stable band. If it fails, the formula becomes a diagnostic only, and PGD becomes the main measure.

14.3 Equality versus threshold

Our formula demands the monitor reading stay *exactly* the same. A real monitor only needs to stay below its alarm threshold, so real evasion could be cheaper. This makes our OC a conservative (upper) estimate of the true cost.

14.4 Saturated concept and model mismatch

Cats vs dogs is too easy (AUROC ≈ 1). We are moving to coco:bottle and later a safety concept. Stage 0 and Stage 1 must use the same model (Qwen2.5-VL-3B in bf16, on the cluster).

15 · Decision rules and next steps

15.1 Pre-registered decision table (rank 1, with confidence intervals)

Outcome	Meaning	Next
Tax falls under compression **and** the behaviour-specific part is negative	Compression weakens monitors beyond generic geometry (original hypothesis)	Build Stage 2 (pixel PGD) for that method
Tax falls; specific ≈ 0; bypass negative	The behaviour moves to places the last-token monitor does not read	Claim is about *where* monitors read; test multi-position monitors
Tax rises; specific negative	Coverage dominates, with behaviour-specific erosion underneath	State the claim at fixed coverage, not fixed rank
Tax and full-capacity cost ≈ flat; OC moved by m or $\|g\|$	No auditability effect; a robustness result	Reframe as a robustness paper (your decision)
First-order check fails	Formula not realised in the true model	Stage 2 required before any OC claim

Current status: the data do **not** meet the first row. A full Stage 2 is therefore not next, but the mini PGD instrument check is mandatory in any case.

15.2 Immediate next steps, in order

1. Analyse the saved SmolVLM Stage 1 results offline (no GPU): rank-1 headline, matched placebo, bypass/specific split, full-capacity cost, incidental detection.
2. Add the fixed-space OC (pre-compression patch embeddings); report both spaces.
3. Collect cluster runs: Qwen2.5-VL-3B Stage 0 + Stage 1 in bf16, on cats and coco:bottle.
4. Run the mini PGD instrument check.
5. Apply the decision table; then expand models, add a safety concept, and build the predictive rule.

15.3 First principles we hold ourselves to

1. Every metric answers a defined question (an optimisation or a probability).
2. The monitor is the object of study; every claim names its rank.
3. Separate the causes: information, sensitivity, monitor tax.
4. Paired design, nulls, bootstrap confidence intervals.
5. One fixed measuring space across all budgets.
6. Validate the instrument against a real attack before interpreting it.
7. Do not pre-commit to a direction.

16 · The longer-term goal: compression under a safety constraint

The project's name, *Compress under a Safety Constraint*, describes the eventual method: compress as much as possible while guaranteeing a safety floor. Following your earlier direction, this stage is formulated with ADMM. It is **deliberately on hold** until the relation between compression and auditability is understood, because an optimiser can only trade compute for safety if the trade actually exists.

16.1 The constrained problem

$$\text{minimise } J_{\text{comp}}(\theta) \;\; \text{subject to} \;\; S(\theta) \geq \delta, \;\; \text{Acc}(\theta) \geq \text{Acc}_{\min} \tag{16.1}$$

J_{comp} is the compute cost (tokens kept), S is the safety score (now OC or its monitor tax), δ is the required safety floor.

16.2 ADMM in simple terms

ADMM (Alternating Direction Method of Multipliers)[34] splits a hard problem into two easier ones linked by an agreement constraint. We keep two copies of the adjustable parameters: θ (optimised for low compute) and z (forced to be safe), with θ = z:

$$L_\rho(\theta, z, u) = J_{\text{comp}}(\theta) + 1_{\{S \geq \delta\}}(z) + (\rho/2)\,\|\theta - z + u\|^2 - (\rho/2)\,\|u\|^2 \tag{16.2}$$

Then repeat three steps: (1) improve θ by ordinary gradient descent; (2) project z onto the safe set {S ≥ δ}; (3) update u by the remaining disagreement θ − z. The quadratic term $(\rho/2)\|\cdot\|^2$ acts like a spring that keeps the two copies close, which gives smoother training than a simple penalty or PID-controlled Lagrangian.[35] At convergence, ρu equals the **shadow price of safety**: how much extra compute one more unit of safety costs.

16.3 Theory we plan to borrow

- Convergence of ADMM for non-convex problems.[36,37]
- The constrained-alignment result of CAID:[38] a learned constrained solution is near-optimal up to a "parametrisation gap" ν, with error growing like √ν.
- A feasibility ceiling: below some token budget, no schedule can meet the safety floor. Originally this came from the Welch bound; our new findings mean the ceiling must be re-derived from the coverage-versus-interference picture.

Closest prior work is robustness-aware compression (ATMC)[39] and ADMM-based pruning,[40] which constrain input-space robustness or sparsity, not monitor-based auditability.

17 · Questions for the Professor

1. **The reframe.** Are you comfortable moving from "compression erodes auditability" to "compression trades robustness for monitorability, with a predictable sign"? Is that a strong enough contribution for ICML?
2. **A fair monitor across budgets.** Should we compare monitors at a fixed rank r, or at a fixed share of explained variance? These can give opposite signs.
3. **Threat model.** Should the headline attacker perturb pixels or embeddings? Should the defender be retrained per budget (refit, current headline) or frozen?
4. **Monitor constraint.** Is "monitor exactly unchanged" acceptable, or should we move to the threshold (inequality) version, which is more realistic but harder to compute?
5. **Where to measure packing.** Block-level packing shows nothing. Should we drop it, or measure at the read position only?
6. **Effect size.** What size of change in attack cost matters in deployment? This should replace the unjustified threshold $\beta \geq 0.25$ from the proposal.
7. **Venue and timeline.** CVPR 2027 (deadline ~13 Nov 2026) is not realistic for a full paper with the current evidence. Do you agree to target ICML 2027 (late January), possibly with a workshop paper first?
8. **Compute.** Can we get priority on the H100 cluster for the Qwen-3B bf16 runs and the PGD check?

18 · Glossary

Term	Meaning
Visual token	A vector representing one image patch inside the model
Keep fraction κ	Share of visual tokens kept; $N = \kappa N_o$
Feature	A property the model tracks, stored as a direction
Superposition	Storing more features than dimensions by letting directions overlap
Welch bound	Minimum possible worst-case overlap when packing F directions into m dimensions
Participation ratio (PR), r90	Effective number of dimensions used; axes needed for 90% of variance
Behaviour score b, margin m	Logit difference between two answers; its size
Gradient g	Sensitivity of b to each input number
Linear probe, rank r	A monitor that reads r directions of the hidden state
Capture (cap)	Share of the behaviour gradient the monitor can see
Monitor tax T	$1/\sqrt{(1-\text{cap})}$: factor by which the monitor raises attack cost
Obfuscation cost OC	Smallest change that flips the answer while the monitor stays still: $m \cdot T/\|g\|$
Matched placebo	Random direction with the same capture; a null for "generic geometry"
Bypass / specific	Parts of the alignment change from unread routes / special to this behaviour
AUROC, d′	Scores for how well a detector separates two classes
PGD	Projected gradient descent, the standard iterative attack
Gradient masking	Misleading gradients that make a model look falsely robust
Refit / frozen; staleness	Monitor retrained per budget / reused; the gap between them
Slope β	$d \log X / d \log \kappa$; $\beta > 0$ means X falls under compression
ADMM	Optimisation method that splits a constrained problem into easier alternating steps

References

1. N. Elhage et al. Toy Models of Superposition. Transformer Circuits Thread, 2022. arXiv:2209.10652.
2. L. R. Welch. Lower Bounds on the Maximum Cross Correlation of Signals. IEEE Trans. Information Theory 20(3):397–399, 1974.
3. E. Stevinson, L. Prieto, M. Barsbey, T. Birdal. Adversarial Attacks Leverage Interference Between Features in Superposition. ICML 2026. arXiv:2510.11709.
4. L. Gorton, O. Lewis. Adversarial Examples Are Not Bugs, They Are Superposition. arXiv:2508.17456, 2025.
5. Y. Liu et al. Superposition Yields Robust Neural Scaling. arXiv:2505.10465, 2025.
6. A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu. Towards Deep Learning Models Resistant to Adversarial Attacks. ICLR 2018.
7. S.-M. Moosavi-Dezfooli, A. Fawzi, P. Frossard. DeepFool: A Simple and Accurate Method to Fool Deep Neural Networks. CVPR 2016.

8. N. Carlini, D. Wagner. Towards Evaluating the Robustness of Neural Networks. IEEE S&P 2017.
9. A. Athalye, N. Carlini, D. Wagner. Obfuscated Gradients Give a False Sense of Security. ICML 2018.
10. N. Carlini et al. On Evaluating Adversarial Robustness. arXiv:1902.06705, 2019.
11. L. Chen et al. An Image is Worth 1/2 Tokens After Layer 2 (FastV). ECCV 2024. arXiv:2403.06764.
12. Y. Zhang et al. SparseVLM: Visual Token Sparsification for Efficient VLM Inference. ICML 2025.
13. S. Yang et al. VisionZip: Longer is Better but Not Necessary in Vision Language Models. CVPR 2025.
14. S. R. Alvar, G. Singh, M. Akbari, Y. Zhang. DivPrune: Diversity-based Visual Token Pruning. CVPR 2025.
15. D. Bolya et al. Token Merging: Your ViT but Faster. ICLR 2023.
16. HiddenDetect: Detecting Jailbreak Attacks against Multimodal LLMs via Monitoring Hidden States. ACL 2025. arXiv:2502.14744.
17. A. Arditi et al. Refusal in Language Models Is Mediated by a Single Direction. NeurIPS 2024.
18. L. Bailey et al. Obfuscated Activations Bypass LLM Latent-Space Defenses. ICLR 2025. arXiv:2412.09565.
19. G. Alain, Y. Bengio. Understanding Intermediate Layers Using Linear Classifier Probes. arXiv:1610.01644, 2016.
20. H. Liu, C. Li, Y. Li, Y. J. Lee. Improved Baselines with Visual Instruction Tuning (LLaVA-1.5). CVPR 2024.
21. S. Bai et al. Qwen2.5-VL Technical Report. arXiv:2502.13923, 2025.
22. A. Marafioti et al. SmolVLM: Redefining Small and Efficient Multimodal Models. arXiv:2504.05299, 2025.
23. J. A. Hanley, B. J. McNeil. The Meaning and Use of the Area under a ROC Curve. Radiology 143(1):29–36, 1982.
24. K. Imai, L. Keele, D. Tingley. A General Approach to Causal Mediation Analysis. Psychological Methods 15(4):309–334, 2010.
25. G. Xiao et al. Efficient Streaming Language Models with Attention Sinks. ICLR 2024.
26. T. Darcet, M. Oquab, J. Mairal, P. Bojanowski. Vision Transformers Need Registers. ICLR 2024.
27. F. Tramèr, N. Carlini, W. Brendel, A. Madry. On Adaptive Attacks to Adversarial Example Defenses. NeurIPS 2020.
28. J. Elson, J. Douceur, J. Howell, J. Saul. Asirra: A CAPTCHA that Exploits Interest-Aligned Manual Image Categorization. ACM CCS 2007 (source of the cats vs dogs images).
29. T.-Y. Lin et al. Microsoft COCO: Common Objects in Context. ECCV 2014.
30. Y. Gong et al. FigStep: Jailbreaking Large Vision-Language Models via Typographic Visual Prompts. arXiv:2311.05608, 2023.
31. X. Liu et al. MM-SafetyBench: A Benchmark for Safety Evaluation of Multimodal LLMs. ECCV 2024.
32. B. Efron, R. Tibshirani. An Introduction to the Bootstrap. Chapman & Hall, 1993.
33. Token-pruning and robustness studies as recorded in our project notes: an ICML 2026 token-pruning safety paper, and arXiv:2607.22716. Full bibliographic details to be verified before submission.
34. S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein. Distributed Optimization and Statistical Learning via the ADMM. Foundations and Trends in ML 3(1):1–122, 2011.
35. A. Stooke, J. Achiam, P. Abbeel. Responsive Safety in RL by PID Lagrangian Methods. ICML 2020.
36. Y. Wang, W. Yin, J. Zeng. Global Convergence of ADMM in Nonconvex Nonsmooth Optimization. J. Sci. Comput. 78:29–63, 2019.
37. M. Hong, Z.-Q. Luo, M. Razaviyayn. Convergence Analysis of ADMM for a Family of Nonconvex Problems. SIAM J. Optim. 26(1):337–364, 2016.
38. B. Zhang et al. Alignment of Large Language Models with Constrained Learning (CAID). NeurIPS 2025. arXiv:2505.19387.
39. S. Gui et al. Model Compression with Adversarial Robustness: A Unified Optimization Framework (ATMC). NeurIPS 2019.
40. T. Zhang et al. A Systematic DNN Weight Pruning Framework using ADMM. ECCV 2018.

Note: 2025–2026 preprints were checked during project planning; venues should be re-verified against primary sources before any submission.